

Seminar/kodeøkt - SOK3023

Maskinlæring for økonomer

Laget av Markus J. Aase

January 14, 2026

Dette dokumentet inneholder en steg-for-steg veiledning for hvordan man kan bygge et nevralt nettverk ved hjelp av TensorFlow ved hjelp av et datasett kalt `adults.csv`. Datasettet er tilgjengelig på fagets GitHub-side under "Oppgaver".

Vi skal prøve å lage en binær klassifiseringsmodell som sier om en person har en lønn som er *under eller lik 50K*, eller *over 50K*.

1. **Last inn bibliotek** Først importerer vi de nødvendige bibliotekene:

```
import tensorflow as tf          # ML bibliotek
from tensorflow import keras     # API for NN
import numpy as np              # Numpy
import matplotlib.pyplot as plt # Plotting
import pandas as pd             # Pandas for dataframes
from sklearn import preprocessing # SciKit Learn
```

2. **Laster inn datasett**

For å laste inn datasettet `adults.csv` i Python, kan vi bruke `pandas`-biblioteket:

```
# Laster inn datasettet fra en CSV-fil
filnavn = "adults.csv"

# Tittel til kolonner
header = [
    'age', 'workclass', 'fnlwgt', 'education', 'education-num',
    'marital-status', 'occupation', 'relationship', 'race', 'sex',
    'capital-gain', 'capital-loss', 'hours-per-week',
    'native-country', 'salary'
]
df = pd.read_csv(filnavn, names = header)

# Viser de første radene i datasettet
print(df.head())
```

Koden over gjør følgende:

- Leser inn `adults.csv` ved hjelp av `Pandas` biblioteket.
- Skriver ut de første radene i datasettet med `df.head()`.

3. Formatet og shape av dataene

Man bør ofte sjekke dimensjonene til datasettet, altså antall rader og kolonner.

```
df.shape
```

Vi kan også sjekke hvilke datatyper de ulike kolonnene er.

```
df.dtypes
```

4. Behandling av manglende verdier

Noen verdier i datasettet kan være manglende eller ukjente (ofte representert som "?"). Ofte ser vi *NaN*, som betyr **Not A Number**, som betyr at det ikke er noe verdi der. Disse må vi håndtere:

```
df = df.replace('?', np.nan) # Erstatter "?" med NaN
df[pd.isnull(df).any(axis=1)].shape # Finn ut hva denne gjør!
```

```
# Kvitter oss med radene som inneholder NaN verdier.
# inplace=True, gjør at vi endrer df direkte!
df.dropna(inplace=True)
```

Prøv å sjekk `df.shape` nå - hva har skjedd nå?

```
df.shape
```

Fortsettelse av prosessering av data

Her velger vi å fjerne kolonnen **education-num**, da den inneholder numeriske representasjoner av utdanningsnivåer, og vi allerede har en annen kolonne, **education**, som inneholder de samme opplysningene i tekstformat. Vi bruker **drop()**-metoden for å slette kolonnen fra DataFrame.

```
df.drop('education-num', axis=1, inplace=True)
```

Vi definerer en liste over de kolonnene som inneholder kategoriske verdier, det kan vi finne av funksjonen **df.dtypes**.

Disse må vi senere konvertere til numeriske verdier for at de skal kunne brukes i maskinlæringsmodeller.

```
categorical_columns = ['workclass', 'education', 'marital-status',
                       'occupation', 'relationship',
                       'race', 'sex', 'native-country']
label_column = ['salary']
```

Følgende funksjon går gjennom en liste med kolonner og skriver ut hvor mange unike verdier hver kolonne har, samt hvilke verdier som finnes. Dette er nyttig for å forstå datasettet før vi foretar videre prosessering av vår DataFrame.

```
def show_unique_values(columns):
    for column in columns:
        uniq = df[column].unique().tolist()
        print(column+ "has"+ str(len(uniq)) +"values" + ":" + str(uniq))
show_unique_values(categorical_columns)
show_unique_values(label_column)
```

5. Prosessere dataene

Her definerer vi en funksjon som konverterer verdiene i en kolonne til heltall. Vi oppretter en dictionary/ordbok (**dic**) der hver unike kategori får en numerisk verdi. Deretter mapper vi denne dictionary'en til DataFrame, slik at verdiene erstattes med sine respektive tallverdier.

```
def convert_to_int(columns):
    for column in columns:
        unique_values = df[column].unique().tolist()
        dic = {}
        for indx, val in enumerate(unique_values):
            dic[val]=indx
        df[column] = df[column].map(dic).astype(int)
        print(column + " done!")
convert_to_int(label_column)
show_unique_values(label_column)
```

Her konverterer vi altså `label_column`, som består av '`<=50K`' og '`>50K`', til henholdsvis **0** og **1**.

6. **One hot encoding** One-hot encoding er en teknikk for å konvertere kategoriske variabler til numeriske verdier på en måte som bevarer informasjonen uten å introdusere en kunstig rangering. Vi bruker dummies for å lage binære kolonner for hver kategori, og deretter fjerner vi de opprinnelige kategoriske kolonnene fra DataFrame.

”Lynkurs” i One-Hot Encoding

Mange maskinlæringsmodeller fungerer best med numeriske verdier, men ofte har datasett kategoriske variabler (f.eks. yrke, bostedsland, kjønn). En måte å konvertere disse til numeriske verdier på er **one-hot encoding**.

One-hot encoding innebærer at vi oppretter en ny binær kolonne for hver unike kategori i en variabel. Hvis en observasjon tilhører en bestemt kategori, settes verdien til 1 i den tilsvarende kolonnen, og ellers settes den til 0.

For eksempel, dersom vi har en kolonne **Color** med verdiene **Red**, **Blue** og **Green**, vil one-hot encoding føre til en transformasjon, se Table 1.

One-hot encoding fjerner problemet med å tildele vilkårlige tallverdier til kategoriske variabler, noe som kan introdusere feilaktige sammenhenger i modellen. Vi bruker **pandas** til å utføre one-hot encoding, på vårt datasett, på følgende måte:

Color	Red	Blue	Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1
Red	1	0	0
Blue	0	1	0

Table 1: Eksempel på one-hot encoding

```
# One hot encoding funksjon
def convert_to_onehot(data, columns):
    dummies = pd.get_dummies(data[columns])
    data = data.drop(columns, axis=1)
    data = pd.concat([data, dummies], axis=1)
    return data
```

Nå kan vi endre dataen ved å ”ta i bruk” one-hot funksjonen:

```
df = convert_to_onehot(df, categorical_columns)
df
```

Prøv å nå se på `df.shape` og `np.sum(df['salary'])`. Hva ser du?

7. **Normaliser data** Nå skal vi normalisere de numeriske kolonnene. De numeriske kolonnene er:

- *age*
- *fnlwgt*
- *capital_gain*
- *capital_loss*
- *hours_per_week*

Dette kan dere bekrefte selv ved å kjøre koden under:

```
df.dtypes
```

Normalisering av numeriske kolonner

I denne delen av koden normaliserer vi de numeriske kolonnene i datasettet for å gjøre dem skalerbare for maskinlæringsmodeller.

```
normalize_columns = ['age', 'fnlwgt', 'capital-gain',
                    'capital-loss', 'hours-per-week']
```

Først definerer vi en liste, `normalize_columns`, som inneholder navnene på de kolonnene som skal normaliseres. I dette tilfellet inkluderer vi kolonnene `age`, `fnlwgt`, `capital-gain`, `capital-loss` og `hours-per-week`.

Deretter definerer vi en funksjon som viser minimum, maksimum, gjennomsnitt og varians for de valgte kolonnene. Denne funksjonen brukes til å undersøke verdiene i de numeriske kolonnene før og etter normalisering.

```
def show_values(columns):
    for column in columns:
        max_val = df[column].max()
        min_val = df[column].min()
        mean_val = df[column].mean()
        var_val = df[column].var()
        print(column + ': values=[ ' + str(min_val) + ', ' + str(max_val) + ' ],
              mean=' + str(mean_val) + ', var=' + str(var_val))
```

NB: Den litt rare "u"-en over, er bare mellomrom i koden. Ble sånn her:)

Funksjonen `show_values` tar en liste med kolonner som argument, og for hver kolonne beregner den minimumsverdien, maksimumsverdien, gjennomsnittet og variansen. Den skriver deretter ut denne informasjonen for å gi oss innsikt i dataene før normalisering.

```
show_values(normalize_columns)
```

Ved å kalle funksjonen `show_values(normalize_columns)` får vi statistikken for de numeriske kolonnene i `normalize_columns`. Dette gir oss et bilde av verdiene før vi anvender normalisering.

Deretter definerer vi en funksjon som normaliserer de valgte kolonnene. Vi bruker en standard skaleringsmetode som er tilgjengelig i `preprocessing`-modulen fra `scikit-learn`, kalt `StandardScaler`.

```
def normalize(columns):
    scaler = preprocessing.StandardScaler()
    df[columns] = scaler.fit_transform(df[columns])

normalize(normalize_columns)
```

I funksjonen `normalize` oppretter vi et `StandardScaler`-objekt. Deretter bruker vi `fit_transform` på de spesifiserte kolonnene for å standardisere verdiene i hver av dem. Denne metoden skalerer dataene slik at de får et gjennomsnitt på 0 og en standardavvik på 1.

Ved å kalle `normalize(normalize_columns)` normaliserer vi kolonnene i `normalize_columns`. Etterpå kaller vi `df.head()` og ser at vi har fått normaliserte verdier i vår `df`.

```
df.head()
```

8. Splitte data inn i training, validation og test sett

Som vi har diskutert flere ganger i kurset, må vi splitte dataen vår. Dette er det maaange måter å gjøre på, vi bruker SciKit Learn biblioteket, som under:

```

from sklearn.model_selection import train_test_split

x_data = df.drop('salary',axis=1)
y_labels = df['salary']

X_train, X_test, y_train, y_test = train_test_split(x_data,y_labels,
test_size=0.2,shuffle=True)

print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)

```

```
X_train.shape[1:]
```

Uttrykket `X_train.shape[1:]` brukes for å hente dimensjonene til en matrise eller DataFrame, bortsett fra den første dimensjonen (antall rader).

- `X_train.shape` returnerer en tuple som beskriver dimensjonene til `X_train`, for eksempel (100, 20) for 100 rader og 20 kolonner.
- `X_train.shape[1:]` skiver bort den første dimensjonen (antall rader) og returnerer en tuple med de gjenværende dimensjonene. For eksempel, hvis `X_train.shape` er (100, 20), vil `X_train.shape[1:]` være (20,) som representerer antall kolonner.

```
X_train
```

Nå kan det være nyttig å se på `X_train` - før vi skal lage modellen vår.

9. Maskinlæringsmodellen - Multi Layer Perceptron

```

model = tf.keras.models.Sequential([
    tf.keras.layers.Input(shape=X_train.shape[1:]),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(16, activation='sigmoid'),
    tf.keras.layers.Dense(16, activation='sigmoid'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

```

Her har jeg definert en modell for dere, og dette har vi gått grundig igjennom. Her skal dere få leke dere som ML-ingeniører:)

Så er det ofte kjekt å kjøre følgende kode:

```
model.summary()
```

Hvorfor får vi disse verdiene når vi kjører `model.summary()`???

10. Definerer optimizer, loss-funksjon og evalueringsmetrikk

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

Nå er vi klare til å *fitte* modellen vår.

```
model.fit(X_train, y_train, epochs=10)
```

Nå evaluerer vi modellen vår.

```
# model.evaluate gir oss informasjon fra Tensorflow Keras.
# verbose = 0: ingen utskrift
# verbose = 1: viser fremdrift av evaluering for hver batch (epoch).
# verbose = 2: viser sammendrag til slutt.
model.evaluate(X_test, y_test, verbose=2)
```

11. Her lager vi en confusion matrix, for å evaluere om modellen vår er bra, dårlig eller midt i mellom:)

Her tester vi modellen vår, og skriver ut confusion matrix'en. Accuracy'en vet vi allerede noe om. Utforsk andre metrikker selv!

Tips: bruk SciKit Learn til å evaluere modellen med andre metrikker.

```
# 1. Prediker modellen til test data
y_pred = model.predict(X_test)
# Konverter sannsynligheter til binaere labels (0 or 1)
y_pred_classes = (y_pred > 0.5).astype(int)

# 2. Generer en confusion matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_classes)
print("Confusion_Matrix:")
print(cm)
```